

Non-Graded Project: Real-World LangChain Deployment Audit

Description

In this capstone project, you'll step into the role of an AI infrastructure consultant for **NovaMind AI**, a startup building an intelligent document-summarization assistant for enterprise users. Their prototype currently runs as a single monolithic LangChain app. Before launching a public beta with thousands of users, they need you to:

- **Design** a modular LangChain microservice architecture
- **Deploy** it using containers and Kubernetes

- **Observe** it with telemetry
- **Stress-test** it with chaos experiments

You'll combine everything from this course-microservice design, Docker + Kubernetes deployment, OpenTelemetry + Prometheus/Grafana observability, and Chaos Mesh-driven resilience testing-to deliver a concise, professional deployment audit.

Learning Objectives

By the end of this project, learners will be able to:

- **Design** a microservice architecture for a LangChain-based summarization system using clear boundaries (retriever, LLM, post-processor) and gRPC communication.
- **Implement and document** a containerized deployment using Docker, AWS ECR, and Kubernetes manifests with appropriate health checks and configuration.



- **Instrument and interpret** telemetry using OpenTelemetry, Prometheus, and Grafana to monitor latency, request rates, and error patterns.
- **Execute and analyze** chaos experiments (pod delete, latency spike, etc.) to measure MTTD and MTTR and assess system resilience.
- **Synthesize findings** into diagrams, dashboards, and a written reflection suitable for technical stakeholders.



Assignment Components

Introductory Slide

Title:

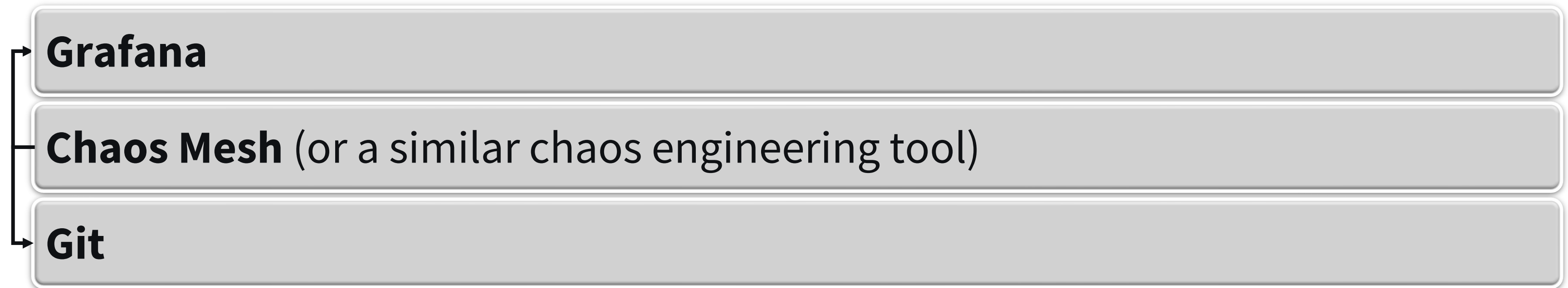
NovaMind AI Launch Check: Is Your LangChain Deployment Ready for the Real World?

Subtitle:

Design, deploy, observe, and break your system-before your users do.

Tools to be used:

- **LangChain** (Python or JS)
- **gRPC + Protocol Buffers**
- **Docker**
- **AWS ECR** (or equivalent container registry)
- **Kubernetes + kubectl**
- **OpenTelemetry** (SDK + Collector, as applicable)
- **Prometheus**



Scenario

NovaMind AI is preparing to launch an intelligent document-summarization assistant for enterprise customers.

Right now, their system is:

- A single **monolithic LangChain app** that fetches documents, calls an LLM, and formats summaries in one process
- Running on a single VM, with no clear boundaries, no health checks, and very limited observability

They are days away from a **public beta** with thousands of concurrent users.

Investors and early customers are asking tough questions:

- How will this scale?
- What happens when the retriever or LLM API slows down or fails?
- Can you prove the system will recover quickly from incidents?

You have been contracted to design and execute a **Real-World LangChain Deployment Audit**. Your task is to propose and demonstrate a production-ready microservice architecture, show how it behaves under load and failure, and present evidence-metrics, diagrams, and logs-that it's ready (or not) for launch.

Instructions for Learners

Step 1 – Define Microservice Boundaries & Architecture

→ **Review** the NovaMind scenario and identify core components: retriever, LLM gateway, post-processor.

→ **Propose** a microservice architecture: which components become separate gRPC services and how they communicate.

→ **Sketch** a one-page architecture diagram showing:

→ Services (retriever, LLM service, summarizer/post-processor)

→ gRPC or HTTP interfaces

Supporting systems (vector store, database, cache, telemetry stack)

Write a short explanation ($\approx$ 200–300 words) justifying your boundaries and communication flow.

Outcome: Drafted architecture diagram + textual justification for NovaMind's new microservice layout.

Step 2 – Containerize & Deploy the Core Service

→ **Select** one core component to fully implement (recommended: retriever or summarizer service).

→ **Create** a production-minded Dockerfile using best practices from the course (multi-stage build, non-root user, no secrets baked in).

→ **Tag and push** the image to **AWS ECR** (or a simulated local registry if ECR is not available).

→ **Write** Kubernetes manifests (deployment.yaml and service.yaml) with:

→ Replica count

→ Environment variables (without hard-coded secrets)

Readiness and liveness probes



Deploy the service to a Kubernetes cluster (local or remote) and verify that pods are running and reachable.

Outcome: Working containerized microservice deployed on Kubernetes, documented in your Deployment Report.

Step 3 – Add Telemetry: Traces & Metrics

→ **Instrument** your selected microservice with **OpenTelemetry**:

→ Initialize tracer/metrics provider

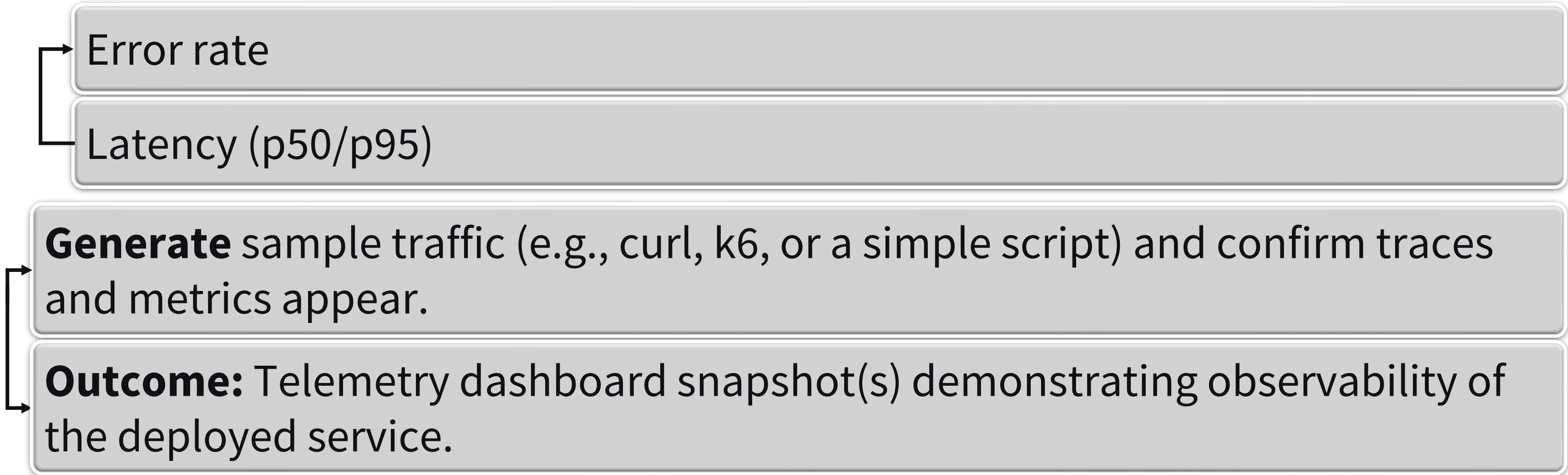
→ Add spans around key operations (e.g., retriever call, LLM request, formatting)

→ **Expose** a metrics endpoint for **Prometheus** (e.g., /metrics).

→ **Configure** Prometheus to scrape your service.

→ **Build** or capture a **Grafana dashboard** showing:

→ Request rate



Error rate

The diagram consists of five horizontal gray boxes with rounded corners. The first two boxes are stacked vertically and contain the text 'Error rate' and 'Latency (p50/p95)' respectively. A bracket on the left side of these two boxes points to the third box. The third box contains the text 'Generate sample traffic (e.g., curl, k6, or a simple script) and confirm traces and metrics appear.' A bracket on the left side of the third and fourth boxes points to the fourth box. The fourth box contains the text 'Outcome: Telemetry dashboard snapshot(s) demonstrating observability of the deployed service.'

Latency (p50/p95)

Generate sample traffic (e.g., curl, k6, or a simple script) and confirm traces and metrics appear.

Outcome: Telemetry dashboard snapshot(s) demonstrating observability of the deployed service.

Step 4 – Run a Chaos Experiment & Measure MTTD/MTTR

- **Design** a small chaos experiment, such as:
 - Deleting one pod of your microservice
 - Injecting 300–500 ms of network latency into the LLM service
- **Define success criteria** before you run it (e.g., MTTD < 60 seconds, MTTR < 5 minutes, error rate spike < 5%).
- **Execute** the experiment using **Chaos Mesh** (or an equivalent tool).
- **Observe** the impact in Jaeger/trace viewer and Grafana dashboards:
 - When do errors start?

When do metrics return to normal?

→ **Calculate** MTTD and MTTR based on timestamps.

→ **Outcome:** A concise chaos experiment log summarizing what failed, how quickly it was detected, and how quickly it recovered.

Step 5 – Synthesize Findings & Reflect

→ **Summarize** your architecture, deployment, telemetry, and chaos testing results into a cohesive story:

→ What works well?

→ Where are the bottlenecks or fragilities?

→ **Reflect** in $\approx$ 500 words on:

→ How your design choices influenced resilience

→ What you would improve if you had more time

→ How you would explain this deployment readiness to NovaMind's CTO

Outcome: A professional-style reflection that ties together architecture, operations, and resilience insights.

Requirement for Submission (AI Grading)

Requirement 1: Architecture Summary

Question Prompt:

“Upload your one-page architecture summary for NovaMind AI, including a diagram and a brief explanation of your chosen microservice boundaries and communication flow.”

Response Type: File Upload

File Types:

- Text / Document: .pdf, .txt, .md
- Image / Diagram: .png, .jpg, .jpeg, .webp
- Coding / Markup: .drawio, .html, .json, .yaml

Requirement 2: Deployment Report (Containerization & Kubernetes)

Question Prompt:

“Upload your Deployment Report containing the Dockerfile, Kubernetes manifests (deployment.yaml, service.yaml), and a brief note (150–250 words) describing how you containerized and deployed the service.”

Response Type: File Upload

File Types:

- Coding / Config: .dockerfile, .yml, .yaml, .json, .py
- Archive for multiple files: .zip, .tar.gz
- Text / Document for the note: .pdf, .txt, .md

Requirement 3: Telemetry Dashboard Snapshot

Question Prompt:

“Provide evidence of your telemetry setup by uploading a screenshot or export of your Prometheus/Grafana dashboards showing latency, request rate, and error metrics for your LangChain service.”

Response Type: File Upload

File Types:

- Image: .png, .jpg, .jpeg, .webp
- Data / Export: .json, .csv
- Document (for brief captions or explanation): .pdf, .txt, .md

Requirement 4: Chaos Experiment Log (Resilience Assessment)

Question Prompt:

“Upload your Chaos Experiment Log summarizing the failure scenario you injected (for example, pod delete or latency spike), how it impacted the system, and the MTTD/MTTR values you measured.”

Response Type: File Upload or Rich Text

File Types:

→ Text / Document: .pdf, .txt, .md, .csv

→ Image (annotated graphs/timelines): .png, .jpg, .jpeg, .webp

Video (short screen recording of the experiment and dashboards, max 15 min): .mp4, .mov, .webm

Requirement 5: Professional Reflection

Question Prompt:

“In about 500 words, reflect on your Real-World LangChain Deployment Audit. What are the main strengths and weaknesses of your architecture and deployment, and what key lessons about resilience will you apply in future AI microservice projects?”

Response Type: Rich Text or File Upload

File Types:

- Text / Document: .pdf, .txt, .md
- Audio (spoken reflection): .mp3, .wav, .aac, .flac
- Video (spoken reflection, max 15 min): .mp4, .mov, .webm

Share The Project

Document: Save the report in PDF format.

Upload: Share in the “Peer Review” area of the course shell with a brief description.

Instructions for Documenting and Sharing The Project for Peer Review

1. Document the Project

- Use word processing software to create your report.
- Ensure all sections of the project document structure are completed.
- Proofread and edit your report for clarity and accuracy.

2. Share the Project

- Save your report in PDF format.
- Upload your documents to the “Peer Review” area in the course shell.
- Provide a brief description of your project in the submission post.

Instructions for Documenting and Sharing The Project for Peer Review

3. Peer Review

- Review the projects submitted by your peers.
- Provide constructive feedback and comments on their work.